



Team Presentation Kit

A ready-to-read script for a 4-member hackathon presentation — plus cheat-sheets for the questions judges love to ask.

The 30-second pitch: LoanLens tells a bank's call-centre agent, in **one second**, whether a loan applicant *can* qualify — using the banking rule called **FOIR** — and if they can't, it instantly shows the **biggest loan they can get**. It stops agents wasting calls on applicants who were never going to qualify.

1 • Team, order & timing

Total time ~7–8 minutes · four speakers · roughly ~1.5–2 min each. Finish your part, then say the one bridge sentence that names the next speaker.

#	SPEAKER	SECTION
1	[Member 1]	Welcome + the problem + <i>What is FOIR</i> (simple)
2	[Member 2]	Live demo — the product & the wow features
3	[Member 3]	The Core-Java engine — which Java topics, why & where
4	[Member 4]	The connectors (Spring Boot, REST, database, security) + closing

2 • The full script

Speaker 1 Welcome + Problem + What is FOIR

Good [morning/afternoon] everyone. We are **Team [Name]**, and our project is **LoanLens**.

Picture a call-centre agent at a bank. All day, people call asking for loans. The problem is simple but expensive: the agent spends 10–15 minutes collecting details from someone who was **never going to qualify** — a waste of time for the agent *and* the customer.

Banks decide who qualifies using a rule called **FOIR — Fixed Obligation to Income Ratio**. In plain words, FOIR is **the share of your monthly income that already goes to paying loan EMIs**. For example: if you earn ₹1,00,000 a month and already pay ₹47,000 in EMIs, your FOIR is **47%**. Banks approve when this stays **under a limit — usually 50%**.

LoanLens puts this rule at the agent's fingertips. Enter a few numbers and, in one second, you get a clear **Eligible / Borderline / Not-eligible** answer — *before* wasting any time on a full credit check.

↪ *To show you how simple and powerful that is, I'll hand over to **[Member 2]** for a live demo.*

Speaker 2 Live demo

Thanks, [Member 1]. Let me show you LoanLens in action. *(open the website)*

This is our **landing page** — LoanLens is a complete web app. An agent **signs up and logs in** securely. *(log in)*

Here's the **screeners**. I'll enter a real applicant: income **₹85,000**, existing EMIs **₹12,000**, a **₹5,00,000** loan at **10%** for **60 months**. I click **Screen applicant** — instantly the dial sweeps to **26.6%**, turns **green — Eligible**, and gives the agent a **line to read out** to the customer.

Now the clever part. Let me try **₹30,00,000**. Now it's **red — Not eligible, 63.7%**. But instead of just saying "no", LoanLens makes a **counter-offer**: *"the largest loan they qualify for is ₹23,53,000"* — or *"keep the amount, but extend the tenure to 84 months."* One click applies it. **That turns a rejection into a sale.**

I can also drag these **what-if sliders** and the dial moves **live**. One click on **"Compare across all products"** shows which loans they qualify for. And the **Dashboard** logs every screening, with the agent's approval rate and stats.

↪ *To explain the engine behind it and the Java we used, over to **[Member 3]**.*

Speaker 3 The Core-Java engine

Thanks, [Member 2]. The heart of LoanLens is a **pure Java engine** — no framework, just core Java — so it's fast and we could test it thoroughly. Here are the Java concepts we used and **why**:

- We use **primitive types** — `double` for money and ratios, `int` for tenure. Deliberate: the classic bug is using `int` and losing decimals in division. We use `double` everywhere, with **explicit type casting** where needed.
- The verdict is chosen with an **if-else-if ladder**: $\text{FOIR} \leq 50 \rightarrow \text{Eligible}$, $\text{else} \leq 55 \rightarrow \text{Borderline}$, $\text{else} \rightarrow \text{Not eligible}$.
- The message is built with a **switch expression** on an **enum** called `Verdict` — type-safe and clean.
- Money uses **`BigDecimal` + `RoundingMode`** — banker's rounding — so our EMI matches the bank's core systems exactly.
- We used **records** for data like `ScreenInputs` and `ScreenResult` — immutable and concise.
- We **validate zero/negative inputs** before any maths, so it never crashes.
- All rates live in **one central config class** — if RBI changes rates, we edit one place.
- And we proved it with **24 JUnit tests**.

So we've used variables, primitive types, operators, type casting, if-else, switch, enums, records, collections and exception handling — the full core-Java toolkit.

↪ Now **[Member 4]** will explain how all these pieces connect together.

Speaker 4 Connectors + Closing

Thanks, [Member 3]. So how does the Java talk to the browser and the database? Through a few key **connectors**:

- **Spring Boot** is the framework that runs everything — it starts a web server and wires our classes together.
- Our Java logic is exposed as **REST APIs**. When the agent clicks Screen, the browser sends a small **JSON** request using `fetch()` to `/api/screen`; Spring runs the engine and sends JSON back. **That JSON-over-HTTP is the connector between the front-end and the Java.**
- To save data we use **Spring Data JPA with Hibernate** — it maps Java objects to database tables, so we write almost **no SQL**.
- The database is **H2**, reached through a **JDBC** connection pool — it stores users and every screening.
- **Spring Security** handles sign-up and login — passwords are stored safely with **BCrypt** hashing.
- And **Maven** ties all these libraries together as our build tool.

To sum up: LoanLens solves a real banking problem — it saves agents' time, turns rejections into offers, and it's a complete, secure, full-stack app built on solid core Java.

↪ *Thank you — we'd be happy to take your questions.*

3 • Handover cheat-sheet

Just memorize your closing sentence:

- **1 → 2:** *"To show you how simple and powerful that is, I'll hand over to [Member 2] for a live demo."*
- **2 → 3:** *"To explain the engine behind it and the Java we used, over to [Member 3]."*
- **3 → 4:** *"Now [Member 4] will explain how all these pieces connect together."*
- **4 → close:** *"Thank you — we'd be happy to take your questions."*

4 · What is FOIR? (the simple version)

FOIR = Fixed Obligation to Income Ratio. It is *the percentage of monthly income already going to loan EMIs*.

$$\text{FOIR} = (\text{existing EMIs} + \text{the new loan's EMI}) \div \text{monthly income} \times 100$$

- **Example:** income ₹1,00,000, total EMIs ₹47,000 → FOIR = **47%** → under the 50% limit → **Eligible**.
- **Lower is safer** — a high FOIR means most of the salary is already committed.
- It's a **real** term used by SBI, HDFC, ICICI and most Indian lenders. (Internationally it's called **DTI — Debt-to-Income**.)

≤ 50% · Eligible

50–55% · Borderline (review)

> 55% · Not eligible

5 · Core-Java topics we used — what, where & why

JAVA TOPIC	WHERE WE USED IT	WHY
Primitive types (double, int, boolean)	FoirEngine, Money	double keeps money/ratios exact; int for whole months
Type casting	FoirEngine.fmt() — (long) value	Format a threshold cleanly (50.0 → "50")
Operators (arithmetic, relational, logical)	EMI formula & validation	Compute EMI, compare FOIR to limits
if-else-if ladder	FoirEngine.screen()	Choose Eligible / Borderline / Not-eligible
switch expression	FoirEngine.screen()	Build the reason text from the verdict
enum	Verdict	Type-safe outcomes instead of loose strings
record (immutable data)	ScreenInputs, ScreenResult, PolicyConfig, Advice, DTOs	Concise, safe data carriers
Classes & objects / OOP	Entities (AppUser, Screening), services	Encapsulation with getters/setters

JAVA TOPIC	WHERE WE USED IT	WHY
Static utility methods	FoirEngine, Money, Advisor	Pure, stateless logic that's easy to test
BigDecimal + RoundingMode	Money.round2()	Banker's rounding — matches bank ledgers
Math library	Math.pow, log, round, floor	EMI formula & solving tenure
Collections + Generics	List<RateSlab>, Optional<AppUser>, JpaRepository<...>	Type-safe lists & lookups
Exception handling	try/catch on DB save; return-error design	Never crash the verdict on a DB hiccup
String.format / String methods	reason text, email.trim().toLowerCase()	Formatting & normalising input
Interfaces	JpaRepository, UserDetailsService	Plug into Spring's contracts
java.time.Instant, UUID	timestamps & IDs	Unique, sortable records
JUnit 5 testing	FoirEngineTest, MoneyTest, AdvisorTest	24 tests prove the maths

6 · Connectors we used — what connects what, and how

CONNECTOR	WHAT IT CONNECTS	HOW WE USED IT
Spring Boot	Everything	Embedded web server + dependency injection
Spring MVC / REST	Browser ↔ Java	@RestController + @PostMapping expose our logic as HTTP endpoints
fetch() + JSON (Jackson)	Front-end ↔ Back-end	JS sends JSON to /api/screen; Jackson converts Java ↔ JSON
Spring Data JPA + Hibernate	Java objects ↔ DB tables	JpaRepository gives save/find with almost no SQL
JDBC + H2 database	App ↔ storage	Connection pool to an on-disk H2 DB for users & screenings
Spring Security + BCrypt	Login ↔ app	Gates pages, hashes passwords, manages the session
Maven	App ↔ libraries	Build tool + dependency manager

One flow, end to end (great to say aloud):

Agent clicks Screen → `fetch()` sends JSON to `/api/screen` → Spring routes it to `ScreenController` → the service runs the pure `FoairEngine` → the result is saved to H2 through JPA → Jackson turns it back into JSON → the JavaScript animates the dial.

7 · Features that impress in hackathons

- 🎯 **Smart Counter-Offer** — turns a "no" into the biggest loan they *can* get (we invert the EMI formula — real algorithm depth).
- 🇮🇹 **Animated FOIR risk dial** — a live gauge sweeping green → amber → red.
- 🏠 **Live what-if sliders** — drag loan/tenure, the verdict updates instantly.
- 🔄 **Multi-product comparison** — screen against every loan product at once.
- 📊 **Analytics dashboard** — approval rate, average FOIR, eligible/borderline/rejected split.
- 🔑 **Real full-stack** — sign-up/login, hashed passwords, a live database.
- 🎨 **Premium, distinctive design** — warm and editorial, not a template.
- 🛡️ **Handles the tricky watch-outs** — precision, rounding, zero/negative inputs, no hard-coded rates.

8 · Likely judge questions — and your answers

Q: Which Java topics did you use?

Primitive types, type casting, operators, if-else, switch, enums, records, classes/OOP, static utility methods, BigDecimal rounding, collections & generics, exception handling, interfaces, and JUnit tests. (See §5.)

Q: Why Spring Boot and not a plain console program?

The logic is plain core Java. Spring Boot lets a real agent use it in a browser and lets us demo it — with REST APIs, a database, and login — with very little boilerplate.

Q: How does the counter-offer actually work?

We invert the EMI maths. The most the applicant can afford is (50% of income – existing EMIs). From that max EMI we solve the formula backwards for the biggest principal; and we solve it for n to find the tenure that qualifies.

Q: How do you avoid the integer-division / precision bug?

We compute everything in double and only round for display. Money rounding goes through one BigDecimal helper using banker's rounding, matching bank systems.

Q: What happens if income is 0 or negative?

Validation runs before any maths and returns a friendly error code — it never crashes or shows a wrong number.

Q: Is it secure?

Yes — Spring Security gates the app, passwords are BCrypt-hashed (never plain text), and the database file is not pushed to GitHub.

Q: Which database, and can you change it?

H2 (file-based, persists). Because we use JPA, we can switch to MySQL / PostgreSQL / Supabase by changing one config line.

Q: Can rates change without redeploying?

All rates and thresholds are centralized in TariffConfig — one edit updates the whole tool. (Next step: an admin screen to edit them live.)

Q: Is FOIR a real thing?

Yes — it's the standard rule Indian banks and NBFCs use for loan eligibility.

Q: What would you build next?

An AI feature that explains each verdict to the customer in plain language, an admin panel for rates, and more loan products.

9 • Numbers to remember

24 JUnit tests, all passing

3 verdict bands — 50% & 55%

5 loan products, compared in one click

~5 sec full agent workflow

1 config file for all rates

Tip: whoever is nervous should take Speaker 1 or 4 — they're the most scripted. Speakers 2 and 3 just narrate the demo and the tables above.